

Anthropic Claude Messages API vs OpenAI API

Three Structural Differences Every Developer Must Know

Sources: platform.claude.com/docs · developers.openai.com/api/docs

System Prompt Placement

Response Content Structure

`max_tokens`: Required vs
Optional

Introduction

Both Anthropic and OpenAI provide powerful APIs for interacting with large language models. While they serve the same purpose — letting you send a prompt and receive a response — the **structure** of how you call them and how they reply is different in important ways.

These differences matter especially when you are building a **wrapper class** that must support both APIs with the same interface. Ignoring them leads to bugs that are hard to trace — like missing a system prompt, crashing on the response object, or getting a rejected request because a required field was omitted.

This document covers the **three most structurally significant differences**, each illustrated with side-by-side Python code examples sourced directly from both APIs' official documentation.

At a Glance

#	Difference	Anthropic	OpenAI
1	System prompt placement	Top-level system= param	Inside messages[] or instructions=
2	Response content structure	response.content[0].text	response.output_text or choices[0]
3	max_tokens field	Required — API rejects without	Optional — has a default

#1 System Prompt Placement

Where you put the instruction that controls the AI's behaviour

What is the system prompt?

The **system prompt** is the instruction you give the AI to set its personality, role, or rules — before the user says anything. For example: *"You are a helpful coding assistant. Always reply in Python."* Both APIs support this concept, but they differ in where you put it.

Anthropic — system is a top-level parameter

In Anthropic's Messages API, the system prompt is a **separate, top-level field** in the request. It lives outside the messages array entirely. This is a deliberate design choice — Anthropic treats the system prompt as a distinct layer of context, not just another turn.

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_KEY")

response = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024,
    system="You are a helpful coding assistant.", # <-- TOP-LEVEL param
    messages=[
        {"role": "user", "content": "Write a Python hello world"}
    ]
)

print(response.content[0].text)
```

Anthropic: system= sits outside messages[]

OpenAI Chat Completions — system inside messages[]

In OpenAI's Chat Completions API (the older API), the system prompt is placed **inside the messages array** as just another message — but with role set to *"system"*. It must be the first item in the list.

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_KEY")

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful coding assistant."}, # <-- INSIDE
        {"role": "user", "content": "Write a Python hello world"}
    ]
)
```

```
{ "role": "user", "content": "Write a Python hello world" }
]
)

print(response.choices[0].message.content)
```

OpenAI Chat Completions: system role inside messages[]

OpenAI Responses API — dedicated instructions= param

OpenAI's newer Responses API (used with GPT-5.x models) adds a top-level **instructions=** parameter — structurally similar to Anthropic's approach. The user prompt goes in **input=** instead of messages.

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_KEY")

response = client.responses.create(
    model="gpt-5.5",
    instructions="You are a helpful coding assistant.", # <-- TOP-LEVEL, like Anthropic
    input="Write a Python hello world"
)

print(response.output_text)
```

OpenAI Responses API: instructions= at top level

Side-by-side comparison

Anthropic

```
client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024,
    system="You are helpful.", # top-level
    messages=[
        { "role": "user",
          "content": "Hello" }
    ]
)
```

OpenAI

```
client.chat.completions.create(
    model="gpt-4o",
    messages=[
        { "role": "system", # inside list
          "content": "You are helpful." },
        { "role": "user",
          "content": "Hello" }
    ]
)
```

	Anthropic	OpenAI (Chat Completions)
System prompt location	Separate system= parameter	Inside messages[] with role='system'
messages[] contains	Only user/assistant turns	System + user + assistant turns
Can be omitted?	Yes — entirely optional	Yes — just omit the system message

Position enforcement	Not in messages — always first	Must be first message
-----------------------------	--------------------------------	-----------------------

Why it matters for your wrapper class: If you write a universal `chat()` method that accepts a `system` parameter, you must route it differently for each provider. For Anthropic, pass `system=` directly to `messages.create()`. For OpenAI Chat Completions, prepend it to the messages list. Mixing these up causes the system prompt to either be silently ignored or treated as a user message.

Source: platform.claude.com/docs/en/api/messages/create · developers.openai.com/api/docs/guides/text

#2

Response Content Structure

How you extract the model's reply from the API response object

What is the response object?

After you call the API, you get a **response object** back. The model's reply text is somewhere inside this object — but the path to get there is completely different between Anthropic and OpenAI. Getting this wrong is one of the most common bugs when switching between the two APIs.

Anthropic — content block list

Anthropic returns **response.content** — a Python list of **typed content blocks**. Each block has a **type** field. For a regular text reply, the block looks like `{"type": "text", "text": "..."}.` You access the text via **response.content[0].text**. This design allows mixed responses — for example, a text block followed by a tool-use block.

```

response = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=512,
    messages=[{"role": "user", "content": "What is 2+2?"}]
)

# Response structure:
# response.content = [
#   TextBlock(type='text', text='2 + 2 equals 4.')
# ]

text = response.content[0].text # <-- navigate the list
print(text) # Output: 2 + 2 equals 4.

# Also available:
print(response.stop_reason) # "end_turn"
print(response.usage.input_tokens) # prompt tokens used

```

Anthropic: response.content is a typed block list

OpenAI Chat Completions — choices list

OpenAI's Chat Completions API returns **response.choices** — a list of choice objects. Each choice has a **.message.content** attribute which is a **plain string**. In normal usage there is one choice at index 0, giving the familiar path **response.choices[0].message.content**.

```

response = client.chat.completions.create(
    model="gpt-4o",

```

```

messages=[{"role": "user", "content": "What is 2+2?"}]
)

# Response structure:
# response.choices = [
# Choice(message=ChatMessage(role='assistant', content='2 + 2 equals 4.'))
# ]

text = response.choices[0].message.content # <-- plain string
print(text) # Output: 2 + 2 equals 4.

# Also available:
print(response.usage.prompt_tokens) # tokens used

```

OpenAI Chat Completions: response.choices[0].message.content

OpenAI Responses API — output array

OpenAI's newer Responses API uses **response.output** — an array of output items. The SDK provides a convenience property **response.output_text** that collects all text from the output array into one string. The official docs warn: do not assume text is always at output[0].content[0].text — the array can also contain reasoning tokens and tool calls.

```

response = client.responses.create(
    model="gpt-5.5",
    input="What is 2+2?"
)

# Convenience property — recommended by OpenAI docs:
print(response.output_text) # "2 + 2 equals 4."

# Raw output structure:
# response.output = [
# {"type": "message", "role": "assistant",
# "content": [{"type": "output_text", "text": "2 + 2 equals 4.", ...}]}
# ]

# Do NOT always assume output[0].content[0].text — can have tool calls etc.

```

OpenAI Responses API: output_text convenience property

Side-by-side comparison

Anthropic

```
# Anthropic
resp = client.messages.create(...)

# Navigate content block list:
text = resp.content[0].text

# Type of .content:
# list of TextBlock / ToolUseBlock
```

OpenAI

```
# OpenAI Chat Completions
resp =
client.chat.completions.create(...)

# Navigate choices list:
text =
resp.choices[0].message.content

# Type of .content:
# str (plain string)
```

	Anthropic	OpenAI Chat Completions
Response field	response.content	response.choices
Text path	response.content[0].text	response.choices[0].message.content
Data type	List of typed content blocks	Plain string
Multi-part?	Yes — text + tool blocks mixed	No — one string per choice
Token usage	response.usage.input_tokens	response.usage.prompt_tokens

Why it matters for your wrapper class: Your unified chat() method must return a string. That means internally you need different extraction logic per provider. A wrong path like response.content on an OpenAI response returns None. Always test both branches with a live call before assuming your extraction is correct.

Source: platform.claude.com/docs/en/home · developers.openai.com/api/docs/guides/text

#3 **max_tokens: Required vs. Optional**

Whether you must tell the API how long the response can be

What is max_tokens?

max_tokens limits how long the model's reply can be. Every token is roughly $\frac{3}{4}$ of a word. Setting **max_tokens=512** means the reply can be at most ~384 words. Without a limit, a model might generate a very long response — costing you more money and taking more time.

Anthropic — max_tokens is REQUIRED

Anthropic's API **requires** you to always provide **max_tokens**. If you omit it, the API immediately returns a **400 Bad Request** error. This is intentional — Anthropic forces you to be explicit about output limits so you never accidentally generate an unbounded response.

```
# CORRECT — max_tokens is required:
response = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024, # <-- REQUIRED, cannot be omitted
    messages=[{"role": "user", "content": "Tell me a story"}]
)

# WRONG — this will raise an error:
response = client.messages.create(
    model="claude-opus-4-5",
    # max_tokens missing!
    messages=[{"role": "user", "content": "Tell me a story"}]
)

# Error: anthropic.BadRequestError: 400 Bad Request
# {"type":"error","error":{"type":"invalid_request_error",
# "message":"max_tokens: field required"}}
```

Anthropic: omitting max_tokens causes a 400 error

OpenAI — max_tokens is OPTIONAL

OpenAI's APIs do **not** require **max_tokens**. If you omit it, the API uses its own internal default — the model will decide how long to respond. You can add it to control costs and response length, but leaving it out does not cause an error.

```
# CORRECT — max_tokens is optional in OpenAI:
response = client.chat.completions.create(
    model="gpt-4o",
```

```

messages=[{"role": "user", "content": "Tell me a story"}]

# max_tokens not needed — uses model default
)

# ALSO CORRECT — you can still set it to control output:
response = client.chat.completions.create(
    model="gpt-4o",
    max_tokens=1024, # optional, but recommended for cost control
    messages=[{"role": "user", "content": "Tell me a story"}]
)

# For newer Responses API: use max_output_tokens
response = client.responses.create(
    model="gpt-5.5",
    max_output_tokens=1024, # optional parameter name differs too!
    input="Tell me a story"
)

```

OpenAI: max_tokens is optional — no error if omitted

What happens if you forget?

Scenario	Anthropic Result	OpenAI Result
max_tokens omitted	400 Bad Request — field required	Succeeds — uses model default
max_tokens=0	400 Bad Request — must be >= 1	400 Bad Request — invalid value
max_tokens very high	Capped at model's context limit	Capped at model's context limit

Why it matters for your wrapper class: Always include max_tokens in your wrapper's default parameters. This makes your wrapper safe for Anthropic (which requires it) and also good practice for OpenAI (prevents unexpectedly large and expensive responses). A safe default is 1024 or 2048 tokens for most use cases.

Source: platform.claude.com/docs/en/api/messages/create · developers.openai.com/api/docs

Putting It All Together

Here is a minimal wrapper class that correctly handles all three structural differences documented above. Each difference maps directly to a specific code path inside `chat()`.

```
import openai
import anthropic

class LLMClient:
    def __init__(self, provider: str, api_key: str, model: str):
        self.provider = provider
        self.model = model

        if provider == "anthropic":
            self.client = anthropic.Anthropic(api_key=api_key)
        elif provider == "openai":
            self.client = openai.OpenAI(api_key=api_key)
        else:
            raise ValueError(f"Unknown provider: {provider}")

    def chat(self,
            messages: list[dict],
            system: str = None,
            max_tokens: int = 1024) -> str: # [Diff 3] always set max_tokens

        if self.provider == "anthropic":
            kwargs = dict(
                model=self.model,
                messages=messages,
                max_tokens=max_tokens, # [Diff 3] required for Anthropic
            )
            if system:
                kwargs["system"] = system # [Diff 1] top-level param
            resp = self.client.messages.create(**kwargs)
            return resp.content[0].text # [Diff 2] content block list

        elif self.provider == "openai":
            if system:
                messages = [{"role": "system", # [Diff 1] inside messages[]
                    "content": system}] + messages
            resp = self.client.chat.completions.create(
                model=self.model,
                messages=messages,
                max_tokens=max_tokens, # [Diff 3] optional but good practice
            )
```

```

return resp.choices[0].message.content # [Diff 2] choices list

# Usage — identical call regardless of provider
anthropic_client = LLMClient("anthropic", "sk-ant-...", "claude-opus-4-5")
openai_client = LLMClient("openai", "sk-...", "gpt-4o")

reply_a = anthropic_client.chat(
    messages=[{"role": "user", "content": "What is 2+2?"}],
    system="Answer concisely."
)
reply_o = openai_client.chat(
    messages=[{"role": "user", "content": "What is 2+2?"}],
    system="Answer concisely."
)

print(reply_a) # "4."
print(reply_o) # "4."

```

Complete wrapper class handling all three structural differences

Summary

#	Difference	Anthropic	OpenAI	Fix in Wrapper
1	System prompt	system= top-level param	role='system' in messages[]	if system: route to correct location
2	Response text	resp.content[0].text	resp.choices[0].message.content	route per provider in return
3	max_tokens	Required — 400 if missing	Optional — uses default	Always include in wrapper default

Key takeaway: These three differences — system prompt placement, response content structure, and the max_tokens requirement — are the minimum you must handle to build a correct unified wrapper. Every other parameter (temperature, stop sequences, streaming) can be layered on top once these fundamentals are right.